UNIVERSITÀ DEGLI STUDI DI PADOVA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

Comunicazione tra un sistema maglev e dispositivi esterni: implementazione e implicazioni per la didattica sulla Teoria del Controllo

Relatore

Prof. Varagnolo Damiano

Laureando

Manuel Palamin

2066688

ANNO ACCADEMICO 2024-2025

Data di laurea 23/07/2025

A chiunque, in futuro, vorrà contribuire al progetto

Sommario

Questa tesi descrive lo sviluppo di un'interfaccia software che consente l'interazione tra un sistema maglev (*Maggy*) e un computer esterno, con l'obiettivo di migliorare l'accessibilità della didattica della Teoria del Controllo e di aumentare la precisione del sistema. L'interfaccia, progettata come una libreria software, permette lo scambio di dati e comandi attraverso una comunicazione seriale USB, demandando al computer esterno l'elaborazione del controllo e il potenziale futuro *sensor fusion* con dati provenienti da altri sensori esterni. Questo approccio consente una maggiore flessibilità nella progettazione dei controllori e offre un ambiente di sviluppo più accessibile per attività sperimentali e didattiche. La tesi illustra le scelte progettuali, l'implementazione del protocollo di comunicazione e l'integrazione con ambienti esterni. I risultati evidenziano come questa soluzione migliori sia le prestazioni del controllo sia l'efficacia dell'apprendimento pratico.

Abstract

This thesis describes the development of a software interface that enables interaction between a maglev (*Maggy*) system and an external computer, with the goal of improving the accessibility of Control Theory education and increasing the system's precision. The interface, designed as a software library, allows the exchange of data and commands via USB serial communication, delegating to the external computer the control computation and potentially, in the future, the *sensor fusion* using data from additional external sensors. This approach provides greater flexibility in controller design and offers a more accessible development environment for experimental and educational activities. The thesis outlines the design choices, the implementation of the communication protocol, and the integration with external environments. The results highlight how this solution improves both control performance and the effectiveness of hands-on learning.

Indice

1 Introduzione **1**
1.1 Sistema fisico 1
1.2 Comunicazione tra *Maggy* e laptop 2
1.3 Software 2
1.4 GitHub e versioning del software 3

2 Modello del sistema e implementazione dell'interfaccia **5**
2.1 Versione 1 12
2.2 Versione 2 14
2.3 Versione 3 15
2.4 Versione 4 19
2.5 Versione 5 20
2.6 Considerazioni 21

3 Controllore PD ed esempio di utilizzo dell'interfaccia **23**

4 Considerazioni e future implementazioni **27**

Elenco delle figure

2.1 Diagramma temporale dell'interazione tra client e server 10
2.2 Diagramma temporale della comunicazione 10

Elenco delle tabelle

2.1 Time-testing della versione on-board 12
2.2 Richieste in formato JSON esteso 13
2.3 Risposte in formato JSON esteso 13
2.4 Time-testing della versione 1 14
2.5 Richieste in formato JSON compatto 14
2.6 Risposte in formato JSON compatto 15
2.7 Time-testing della versione 2 15
2.8 Struttura dei payload di richiesta 17
2.9 Struttura dei payload di risposta 18
2.10 Time-testing della versione 3 19
2.11 Nuova struttura del payload di risposta di `get_position` 19
2.12 Time-testing della versione 5 21

Capitolo 1

Introduzione

La levitazione magnetica, anche conosciuta come *Maglev*, è una tecnologia per cui un oggetto magnetico riesce a fluttuare nell'aria.
Il principio alla base del suo funzionamento fa leva sulle leggi della fisica. In particolare, viene utilizzata la forza elettromagnetica per contrastare la forza gravitazionale che agisce sull'oggetto.
Questo tipo di tecnologia trova nella realtà svariate applicazioni in diversi ambiti, spaziando dall'ingegneria alla medicina e trovando la sua più nota applicazione nel settore dei trasporti, ovvero i treni *Maglev*. [1]
Data l'intrinseca instabilità fisica di un sistema *Maglev*, affinché questo possa funzionare è necessario adottare opportune leggi di controllo. Inoltre, il fatto che l'oggetto sia sospeso nell'aria offre molteplici gradi di libertà nel suo movimento, rendendo il sistema particolarmente adatto a supportare la didattica nella Teoria del Controllo, una nuova applicazione della tecnologia in questione.
Take-home Maglev lab è un progetto nato presso l'Università Tecnica della Norvegia (NTNU), il cui scopo è quello di migliorare gli attuali dispositivi *Maglev* per la didattica e renderli inoltre maggiormente accessibili. [2]
Nel presente elaborato, viene descritto il contributo dell'autore al progetto, che ha riguardato lo spostamento della computazione del controllore su un computer esterno rispetto al sistema *Maglev* da controllare.

1.1 Sistema fisico

Il sistema fisico è composto da una scheda elettronica su cui sono collegati 4 solenoidi e 4 magneti permanenti, disposti a stella, sopra i quali viene fatto fluttuare un disco magnetico. Sulla scheda, sono distribuiti 8 sensori di Hall, utilizzati per stimare la posizione del disco attraverso

la lettura del campo magnetico sui 3 assi X, Y e Z. I sensori sono multiplexati, così da rendere la lettura dei valori sui 3 assi come se provenisse da un unico sensore.
A partire dal teorema di Earnshaw [3], è possibile asserire che non esiste nessuna possibile configurazione statica di ferromagneti in grado di far levitare stabilmente un oggetto contro la gravità, nemmeno quando le forze magnetiche sono più forti di quelle gravitazionali. Per questo motivo, il sistema è dotato di solenoidi oltre ai magneti permanenti. Il controllo dell'hardware avviene per mezzo di un microcontrollore Teensy 4.1, sul quale esegue una routine di correzione delle correnti che attraversano i solenoidi in base alla posizione stimata del disco. In questo modo, i campi magnetici generati vengono continuamente corretti per mantenere il disco in equilibrio nella posizione desiderata.
Il disco gode di 6 gradi di libertà: 3 traslazionali e 3 rotazionali. Attualmente si sta cercando di affinare la stabilità della levitazione su un punto di equilibrio fisso sull'asse Z, con disco parallelo al piano XY. Tuttavia, in prospettiva futura, si mira a un controllo più sofisticato che consenta al disco di muoversi liberamente nello spazio sfruttando tutti i 6 gradi di libertà.
Attualmente, viene usato un controllore di tipo PD (Proportional-Derivative), ma si sta cercando di affinare la stabilità attraverso tecniche più avanzate, come ad esempio l'MPC (Model Predictive Control).

1.2 Comunicazione tra *Maggy* e laptop

La comunicazione tra il sistema *Maglev* (*Maggy*) e laptop avviene tramite connessione seriale via cavo USB. Sui dispositivi Teensy, la libreria `Serial` di Arduino si basa sul protocollo USB CDC [4]. Ciò implica principalmente tre aspetti:

- Il baud-rate della comunicazione viene ignorato, anche se modificato in maniera forzata con la funzione `Serial.begin();`
- I dati vengono raggruppati in pacchetti da 64-byte anzichè essere organizzati in uno stream di bytes;
- I dispositivi operano attraverso un meccanismo di polling dove gli host fanno richieste di dati ad intervalli fissi di tempo ($\sim 1ms$ per la USB full-speed)

1.3 Software

I principali linguaggi di programmazione utilizzati, sono Arduino (che si basa su C/C++, con funzioni native per l'interazione con l'hardware) per quanto riguarda il Teensy; Python per il programma che viene eseguito sul laptop.

Oltre alle sintassi utilizzate che differenziano a colpo d'occhio i due linguaggi qui sopra, un'altra sostanziale differenza tra i due è data dal fatto che Arudino è un linguaggio compilato, mentre il Python è interpretato.
Nelle versioni precedenti del sistema, il controllore veniva interamente scritto in Arduino ed eseguito a bordo del Teensy (controllo on-board). Al fine di rendere la programmazione più intuitiva e flessibile, si è deciso di migrare verso un'architettura in cui il controllo viene gestito da un programma Python eseguito su un computer esterno.

1.4 GitHub e versioning del software

Il progetto è pubblicamente disponibile su GitHub [5], una piattaforma web basata sul sistema di controllo di versione Git. Il suo principale scopo è quello di facilitare la collaborazione tra sviluppatori e tracciare in modo strutturato l'evoluzione di un progetto software.

GitHub consente di:

- mantenere uno storico completo delle modifiche apportate ai file del progetto;
- gestire versioni parallele del codice attraverso l'uso di branch;
- collaborare con più utenti in modo ordinato e sicuro;
- creare repository pubblici per archiviare e documentare i progetti open-source

Nel contesto dello sviluppo software, l'uso di un sistema di versioning come GitHub garantisce trasparenza, riproducibilità e controllo del codice, semplificando anche il processo di individuazione e correzione di errori introdotti nel tempo. Per questi motivi, si è optato per il suo utilizzo nel progetto in questione.

Capitolo 2

Modello del sistema e implementazione dell'interfaccia

Il principale contributo dell'autore nel progetto, nonchè argomento centrale del presente elaborato, è stato quello di realizzare un'interfaccia software per garantire la comunicazione tra *Maggy* ed altri dispostivi esterni.
Al fine di migliorare la didattica, si è deciso di variare le modalità di programmazione del controllore. L'idea è quella di permettere agli studenti di programmare quest'ultimo in Python sul proprio laptop, cosicché la programmazione risulti più accessibile. Inoltre, si evita così la riscrittura della memoria del Teensy ogni volta che il codice subisce variazioni. Così facendo, si potrà avere in futuro la possibilità di implementare una GUI per rendere ancora più accessibile l'utilizzo del dispositivo. Ciò non sarebbe stato possibile nelle precedenti versioni on-board data l'assenza di schermo e tastiera sul Teensy.
Un'altra ragione per cui si è deciso di implementare suddetta interfaccia, risiede nella possibilità di avere un sistema esterno che coordini più dispositivi contemporaneamente. Detto in altre parole, diventerebbe possibile integrare delle telecamere come sensori di stima della posizione del disco, ma senza collegarle direttamente a *Maggy*, operazione che risulterrebbe essere complicata altrimenti. Dunque, il computer centrale a cui si collegherebbero *Maggy* e le telecamere, avrebbe il compito di fare *sensor fusion* tra i dati rilevati tramite *computer vision* e i dati provenienti dai sensori Hall, in modo da affinare il controllo del magnete.

La comunicazione tra le parti, a livello astratto, può essere descritta da un modello client-server: il computer effettua le richieste, mentre *Maggy* provvede a fornire delle risposte a quest'ultime. Dal punto di vista fisico invece, i due dispositivi sono collegati tramite cavo USB.
Sul client, al fine di implementare un controllore per il sistema *Maglev*, è di principale interesse e necessità svolgere le seguenti operazioni:

- lettura dei valori dei sensori di Hall per la stima della posizione del magnete;

- lettura dei valori delle correnti applicate ai solenoidi;
- modificare i valori delle correnti applicate ai solenoidi in modo da aggiustare la posizione del magnete

Al fine di soddisfare tali richieste, *Maggy* provvede a registrare i dati rilevati dai sensori di Hall, aggiornandoli continuamente e rendendoli sempre pronti alle richieste del client. Ogni registrazione, prevede una sovrascrittura di specifiche variabili ridenominate in maniera significativa. Per una più immediata comprensione di come opera il server, di seguito ne viene riportato il frammento di codice (Arduino) centrale che esegue. Si precisa che il codice fa riferimento all'ultima versione del software. Pertanto, non è detto combaci con il codice eseguito nelle precedenti versioni.

```
// ...
int interval = round(1e6 / 5000); // 5kHz
//...

/*
 *  codice che viene eseguito iterativamente nel server
 */
void loop() {

    // il builtin-led viene mantenuto acceso fintanto che la connessione seriale
    ↪  rimane attiva
    (Serial) ? digitalWrite(LED_BUILTIN, HIGH) : digitalWrite(LED_BUILTIN, LOW);

    currentTime = micros();

    if(currentTime - previousTime >= (unsigned long)interval) {
        unsigned long dt = currentTime - previousTime;
        sensors_sampling(dt);   // campionamento e filtraggio dei valori dei sensori
        checkSensorFreeze();    // reset dei solenoidi nel caso il magnete si sia
        ↪  attaccato alla scheda
        previousTime = currentTime;
    }

    if(Serial.available()) {
        uint8_t cmd = Serial.read();  // il primo byte contiente l'identificatore
        ↪  della richiesta
        processCommand(cmd);          // valuta la richiesta ed esegue determinate
        ↪  operazioni di conseguenza
    }
}
```

A sua volta il client, per poter compiere le proprie operazioni di lettura e modifica dei valori del sistema *Maglev*, necessita di un'interfaccia di interazione con *Maggy* come quella riportata nel codice seguente (Python). Si precisa che pure questo codice fa riferimento all'ultima versione del software, e che vengono illustrate le sole firme dei metodi senza rispettivi corpi per semplicità di lettura.

```python
# ...

# iterfaccia per comunicare con un sistema Maglev via connessione seriale
class MaglevInterface:

    # ...

    # apre la comunicazione seriale
    def _open_comm(self) -> None:
        # ...

    # chiude la comunicazione seriale
    def _close_comm(self) -> None:
        # ...

    # invia un comando al server attraverso la connessione seriale e ritorna la
    ↪  corrispondente risposta ricevuta sulla stessa connessione
    def _send_command(self, cmd_name: bytes, parameters: Optional[Dict[str, Any]] =
    ↪  None) -> Optional[bytes]:
        # ...

    # ritorna i valori dei campi magnetici sui 3 assi (magFieldX, magFieldY,
    ↪  magFieldZ) e le rispettive derivate(dMagFieldX, dMagFieldY, dMagFieldZ)
    def get_position(self) -> Optional[Dict[str, float]]:
        # ...

    # ritorna i valori delle correnti che attraversano i solenoidi
    def read_currents(self) -> Optional[Dict[str, float]]:
        # ...

    # modifica le correnti che attraversano i solenoidi
    def set_solenoid_currents(self, currents: Dict[str, int]) -> bool:
        # ...

    # reset delle correnti sui solenoidi ai valori di default
    def reset_solenoids(self) -> bool:
        # ...
```

```python
# ritorna lo stato del sistema Maglev
def get_status(self) -> Optional[Dict[str, Any]]:
    # ...
```

Si è scelto di creare una classe anzichè una libreria di funzioni per le seguenti ragioni:

- per gestire la connessione seriale, viene usato un oggetto `pyserial` come attributo della classe `MaglevInterface`, evitando la creazione di variabili globali che non rappresentano una buona norma. Permette inoltre di automatizzare la creazione, apertura e chiusura della connessione quando un'istanza della classe viene creata, evitando che il programmatore che uilizza questa interfaccia si dimentichi di aprire e chiudere la connessione;
- possibilità di creare metodi privati e pubblici per gestire al meglio i privilegi di accesso al programmatore che vuole usare l'interfaccia;
- possibilità di avere una connessione separata per ogni istanza della classe, così da facilitare la gestione di molteplici connessioni;
- soluzioni software orientate agli oggetti sono più semplici da mantenere in futuro in caso di modifiche

All'interno dei vari metodi della classe, è stata considerata la possibilità di generazione di errori ed eccezioni runtime. Si è scelto di propagarli alle funzioni chiamanti, delegandone la gestione al programmatore che utilizza l'interfaccia.
Di seguito, vengono descritte nel dettaglio le operazioni che l'API (in riferimento all'ultima versione) permette di svolgere:

- `get_position()` : ritorna un dizionario con dominio delle chiavi 'x', 'y', 'z', 'dx', 'dy', 'dz' contenente i valori dei campi magnetici sui 3 assi e le rispettive derivate;
- `read_currents()` : ritorna un dizionario con dominio delle chiavi 'x+', 'x-', 'y+', 'y-' contenente i valori delle correnti che attraversano i solenoidi;
- `set_solenoid_currents(currents : dict)` : ritorna `True` se modifica avvenuta correttamente;
- `reset_solenoids()` : ritorna `True` se reset avvenuto correttamente;
- `get_status()` : ritorna un dizionario con dominio delle chiavi 'solenoids', 'sensors', 'uptime' contenente rispettivamente i valori di: numero di solenoidi nel sistema, numero di sensori di Hall nel sistema, da quanto tempo è attivo il sistema

Il comando `get_status()` al momento non ha importanti impieghi, ma ne è stata prevista l'implementazione per eventuali utilizzi futuri.
Ogni comando dell'API, al suo interno, sfrutta il metodo `_send_command(cmd_name: bytes, parameters: dict)` per inviare le richieste a *Maggy* e ricevere le corrispondenti risposte che poi vengono elaborate nei rispettivi metodi chiamanti.

Poiché il sistema può essere considerato *real-time*, è fondamentale che la comunicazione avvenga in tempi celeri. Di seguito, viene riportata un'analisi sugli istanti di tempo significativi che descrivono una generica routine di request-response.
Quando il client decide di effettuare una richiesta al server, e non appena esso ha la richiesta pronta all'invio, all'istante t_0 inizia la trasmissione della richiesta. Siccome la ricezione da parte del destinatario non è istantanea, l'istante $t_A > t_0$ indica l'istante di tempo in cui il server riceve il primo byte trasmesso. Poiché una generica trasmissione può coinvolgere l'invio di molteplici bytes, definiamo t_1 l'istante in cui viene inviato l'ultimo byte della richiesta a cui corrisponde l'istante t_B, ovvero l'istante in cui quest'ultimo byte è giunto a destinazione. Possiamo dunque definire $t_{request} = t_B - t_0$.
Una volta che il server ha ricevuto completamente la risposta, deve prima interpretare la richiesta, e sulla base di questa elaborare una risposta. Dunque, definiamo $t_{serviceRoutine} = t_C - t_B$, dove t_C indica l'istante di inizio trasmissione della risposta del server. Solo una volta che la risposta sarà pronta, allora il server provvederà ad iniziare la trasmissione di quest'ultima verso il client. Poichè il procedimento di response è analogo a quello di request visto sopra, possiamo già definire $t_{response} = t_3 - t_C$.
Riassumendo, il tempo che trascorre ogni volta che client e server necessitano di comunicare è pari a $t_{TOT} = t_{request} + t_{serviceRoutine} + t_{response} = t_3 - t_0$. Di seguito vengono riportati degli schemi che facilitano la visualizzazione della sequenza temporale appena descritta.

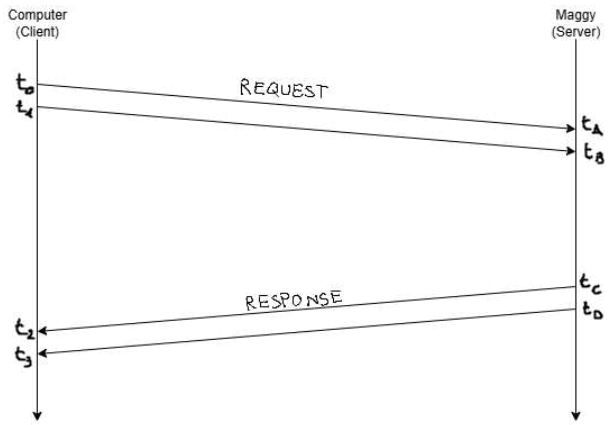

Figura 2.1: Diagramma temporale dell'interazione tra client e server

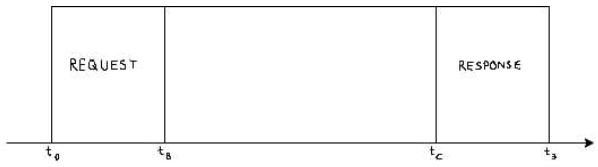

Figura 2.2: Diagramma temporale della comunicazione

In base al tipo di richiesta effettuata dal client, t_{TOT} varia. Più avanti nel presente elaborato ne verranno illustrati i vari valori anche sulla base delle varie versioni implementate.

C'è da ricordare che, per effettuare controllo *real-time* di un sistema come *Maggy*, è fondamentale garantire velocità nella comunicazione tra le parti. Infatti, la sfida principale è stata quella di ridurre il più possibile t_{TOT}.

Durante il ciclo di sviluppo del progetto, sono state realizzate diverse versioni dell'interfaccia e del software intero, ciascuna delle quali ha introdotto nuove funzionalità e miglioramenti rispetto alla precedente. Il principale motivo per cui si è continuato ad indagare su nuove possibili migliorie del software, che hanno portato a versioni multiple dello stesso, è stata la necessità di ottenere sempre più alte prestazioni nella comunicazione tra *Maggy* e computer.
Prima di vedere nel concreto le varie versioni menzionate, viene brevemente illustrato come sono state effettuate le misure di performance dell'interfaccia.

```
# num_iterations variabile tra 5000 e 50000 in base a quanto veloce era la versione

def test_get_position_time(maglev: MaglevInterface, iterations: int =
↪  num_iterations):
    times = []
    for _ in range(iterations):
```

```
        start_time = time.perf_counter()
        maglev.get_position()
        end_time = time.perf_counter()
        times.append(end_time - start_time)

    avg_time = sum(times) / iterations
    avg_frequency = 1 / avg_time
    print(f"Average time for get_position(): {avg_time:.6f} seconds (frequency:
    ↪  {avg_frequency:.2f} Hertz)")


def test_read_currents_time(maglev: MaglevInterface, iterations: int =
↪  num_iterations):
    # ...


def test_set_currents_time(maglev: MaglevInterface, iterations: int =
↪  num_iterations):
    # ...

def test_reset_solenoids_time(maglev: MaglevInterface, iterations: int =
↪  num_iterations):
    # ...

def test_get_status_time(maglev: MaglevInterface, iterations: int = num_iterations):
    # ...

if __name__ == "__main__":
    with MaglevInterface(port) as maglev:
        test_get_position_time(maglev)
        test_read_currents_time(maglev)
        test_set_currents_time(maglev)
        test_reset_solenoids_time(maglev)
        test_get_status_time(maglev)
```

Viene mostrata la sola implementazione della funzione `test_get_position_time(maglev: MaglevInterface, iterations: int)` perchè le altre sono analoghe. Di fatto, il test consiste nel richiamare una funzione in maniera ripetuta, calcolare il tempo che intercorre tra la chiamata a funzione e il suo ritorno, e calcolare una media del tempo che impiegano sue molteplici esecuzioni. I risultati stampati, ovvero i tempi medi di esecuzione delle specifiche routine, sono stati poi usati come indicatori di performance della versione testata. Si riporta qui sotto una tabella riepilogativa dei tempi misurati sulla versione on-board del software (è stato utilizzato del codice con logica analoga a quello sopra), utili

per confrontare quelli ottenuti nelle varie versioni che a breve verranno analizzate. Siccome nell'implementazione del PD controller, che verrà illustrata in seguito, il client ha effettuato solamente le richieste `get_position`, `read_currents` e `set_solenoid_currents`, allora si riportano soltanto le principali misure di interesse.

Tabella 2.1: Time-testing della versione on-board

Comando	t_{TOT} **[**μs**]**
`get_position`	~ 74.98
`read_currents`	~ 71.75
`set_solenoid_currents`	~ 2.74

Di seguito, vengono presentate le varie versioni implementate con corrispondeti analisi e considerazioni.

2.1 Versione 1

Dopo alcune ricerche ed analisi iniziali, e considerate le risorse a disposizione, si è deciso di realizzare una prima versione che sfrutti una comunicazione USB Serial con impacchettamento dei dati tramite JSON.
Date le migliorie del software che hanno portato alla realizzazione di successive versioni, hanno reso questa un prototipo di testing per verificare la corretta comunicazione tra le parti.

JSON (JavaScript Object Notation) è un formato di interscambio dati leggero, leggibile e strutturato, ampiamente utilizzato per lo scambio di informazioni tra client e server, specialmente nelle applicazioni web. Si è deciso inizialmente di impiegarlo per la comunicazione tra *Maggy* e laptop data la sua efficace proprietà di *human-readability*.
In tabella 2.2 e tabella 2.3, vengono riportate rispettivamente le formattazioni JSON delle richieste e delle risposte. Tra le risposte c'è una riga in più, corrispondente al messaggio di risposta inviato in caso di eventuali errori.

Tabella 2.2: Richieste in formato JSON esteso

Comando	**Messaggio JSON**
get_position	{"command": "READ_SENSORS"}
read_currents	{"command": "READ_CURRENTS"}
set_solenoid_currents	{"command": "SET_CURRENTS", "solenoids": [{"pin1": 2, "pin2": 3, "current": value}, {"pin1": 4, "pin2": 5, "current": value}, {"pin1": 6, "pin2": 7, "current": value}, {"pin1": 8, "pin2": 9, "current": value}]}
reset_solenoids	{"command": "RESET_CURRENTS"}
get_status	{"command": "GET_STATUS"}

Tabella 2.3: Risposte in formato JSON esteso

Comando	**Messaggio JSON**
get_position	{"response" : "SENSOR_VALUES", "values" : ["x" : value, "y" : value, "z" : value]}
read_currents	{"response" : "CURRENT_VALUES", "values" : ["X_POS" : value, "Y_NEG" : value, "X_NEG" : value, "Y_POS" : value]}
set_solenoid_currents	{"response" : "ACK_SET_CURRENTS", "status" : "OK"}
reset_solenoids	{"response" : "ACK_RESET_CURRENTS", "status" : "OK"}
get_status	{"response" : "SYSTEM_STATUS", "solenoids" : value, "sensors" : value, "uptime" : value}
error	{"response" : "ERROR", "code" : value, "message" : value}

Risultati

Testando la presente versione con il codice di testing illustrato precedentemente, si ottengono i seguenti risultati.

Tabella 2.4: Time-testing della versione 1

Comando	t_{TOT} **[**μs**]**
`get_position`	~ 15500
`read_currents`	~ 8800
`set_solenoid_currents`	~ 5100

Confrontando i valori con quelli della versione on-board, si notano delle notevoli discrepanze. Questa versione risulta essere infatti troppo lenta, rendendo il controllo del magnete instabile. Il problema principale della lentezza nella trasmissione è dovuto alla presenza di diversi caratteri (bytes) nei messaggi inviati e ricevuti.

2.2 Versione 2

In questa seconda versione, è stato deciso di continuare ad utilizzare il formato JSON per l'impacchettamento dei dati, ma riducendo la dimensione delle chiavi utilizzate per identificare i valori trasmessi. Così facendo, viene ridotta la dimensione in bytes da trasmettere attraverso la connessione seriale e di conseguenza si riducono i tempi di trasmissione.
Nelle seguenti tabelle vengono illustrate le nuove formattazioni di richieste e risposte.

Tabella 2.5: Richieste in formato JSON compatto

Comando	**Messaggio JSON**
`get_position`	`{"cmd": "0"}`
`read_currents`	`{"cmd": "1"}`
`set_solenoid_currents`	`{"cmd": "2", "s": [{"p1": 2, "p2": 3, "c": value}, {"p1": 4, "p2": 5, "c": value}, {"p1": 6, "p2": 7, "c": value}, {"p1": 8, "p2": 9, "c": value}]}`
`reset_solenoids`	`{"cmd": "3"}`
`get_status`	`{"cmd": "4"}`

Tabella 2.6: Risposte in formato JSON compatto

Comando	**Messaggio JSON**
`get_position`	`{"r" : "0", "vls" : ["x" : value, "y" : value, "z" : value]}`
`read_currents`	`{"r" : "1", "vls" : ["X+" : value, "Y-" : value, "X-" : value, "Y+" : value]}`
`set_solenoid_currents`	`{"r" : "2", "s" : "OK"}`
`reset_solenoids`	`{"r" : "3", "s" : "OK"}`
`get_status`	`{"r" : "4", "slnds" : value, "snsrs" : value, "ut" : value}`
`error`	`{"r" : "ERROR", "c" : value, "msg" : value}`

Risultati

Questa seconda versione ha prodotto i seguenti risultati.

Tabella 2.7: Time-testing della versione 2

Comando	t_{TOT} **[**μs**]**
`get_position`	~ 15000
`read_currents`	~ 5600
`set_solenoid_currents`	~ 5100

Confrontando i dati con quelli ottenuti nella versione precedente, non si notano miglioramenti significativi. Infatti, l'utilizzo del formato JSON in questo contesto non è la scelta migliore: sono presenti diversi caratteri che appesantiscono la comunicazione. Si preferisce dunque sacrificare la *human-readability* per ottenere maggior prestazione in termini di velocità. Come spiegato più avanti, dalla versione successiva si è optato per soluzioni alternative alla formattazione JSON.

2.3 Versione 3

Un particolare tipo di packing dei dati, quando si lavora con comunicazioni seriali, è il COBS (Consistent Overhead Byte Stuffing). In particolare, COBS è un algoritmo di codifica dei dati che opera nel seguente modo.

Fase di codifica: L'obiettivo della fase di codifica è garantire che il byte 0x00, utilizzato tipicamente come delimitatore di pacchetto, non compaia mai all'interno dei dati trasmessi. L'algoritmo suddivide il flusso di dati originali in blocchi che non contengono il byte 0x00, e per ciascun blocco inserisce un byte di codice (detto length byte) che indica la lunghezza del blocco più uno. Questo byte viene anteposto al blocco stesso. Ogni volta che si incontra un byte 0x00 nei dati originali, questo non viene trasmesso ma viene sostituito da un nuovo length byte che indica la posizione del prossimo 0x00 (o la fine dei dati). Il risultato finale è una sequenza di byte che non contiene mai 0x00, permettendo di usare proprio questo byte come marcatore sicuro di inizio o fine pacchetto.
Fase di decodifica: Durante la decodifica, il destinatario legge il primo length byte, lo interpreta come la lunghezza del blocco successivo (meno uno) da copiare direttamente nel buffer di uscita. Se il length byte è inferiore a 0xFF, viene reinserito un byte 0x00 alla fine del blocco decodificato, ricostruendo fedelmente la posizione originale degli zeri. Questo processo viene ripetuto fino al termine del pacchetto, ottenendo così la sequenza originale dei dati, inclusi tutti i byte 0x00 che erano stati rimossi e ricodificati.
Ne vengono riportati di seguito i codici corrispondenti.

```python
# algoritmo di codifica
def cobs_encode(data: bytes) -> bytes:
    encoded = bytearray()
    code_ptr = 0  # posizione del code byte
    code = 1      # code value iniziale
    encoded.append(0)  # placeholder per il primo code byte

    for byte in data:
        if byte == 0:  # chiusura del blocco
            encoded[code_ptr] = code
            code_ptr = len(encoded)
            encoded.append(0)  # nuovo placeholder per il code byte
            code = 1
        else:
            encoded.append(byte)
            code += 1
            if code == 0xFF:  # dimensione massima per il blocco raggiunta
                encoded[code_ptr] = code
                code_ptr = len(encoded)
                encoded.append(0)
                code = 1

    encoded[code_ptr] = code  # code byte del blocco finale
    encoded.append(0)  # marcatore finale 0x00
```

```python
    return bytes(encoded)


# algoritmo di decodifica
def cobs_decode(data: bytes) -> bytes:
    decoded = bytearray()
    ptr = 0


    while ptr < len(data) - 1:  # viene ignorato il marker finale 0x00
        code = data[ptr]
        ptr += 1
        for i in range(code - 1):
            decoded.append(data[ptr])
            ptr += 1
        if code < 0xFF:  # controllo della dimensione massima del blocco
            decoded.append(0)

    return bytes(decoded[:-1])  # ignora il 0x00 finale
```

É stato quindi deciso di utilizzare questo tipo di codifica, in alternativa alla formattazione dati JSON, vista la sua predisposizione per flussi di dati sequenziali.

Poichè con COBS si lavora a livello di bytes, è stato scelto di organizzare i payload di richiesta e risposta nel seguente modo.

Tabella 2.8: Struttura dei payload di richiesta

Comando	**Payload**
`get_position`	`| comando x 1 byte |`
`read_currents`	`| comando x 1 byte |`
`set_solenoid_currents`	`| comando x 1 byte | current_on_pins_2-3 x 1 byte | current_on_pins_4-5 x 1 byte | current_on_pin_6-7 x 1 byte | current_on_pins_8-9 x 1 byte |`
`reset_solenoids`	`| comando x 1 byte |`
`get_status`	`| comando x 1 byte |`

Tabella 2.9: Struttura dei payload di risposta

Comando	**Payload**
`get_position`	`| OK x 1 byte | x_sensor_position x 4 byte | y_sensor_position x 4 byte | z_sensor_position x 4 byte |`
`read_currents`	`| OK x 1 byte | x_+_current x 4 byte | y_-_current x 4 byte | x_-_current x 4 byte | y_+_current x 4 byte |`
`set_solenoid_currents`	`| OK x 1 byte |`
`reset_solenoids`	`| OK x 1 byte |`
`get_status`	`| OK x 1 byte | number_of_solenoids x 1 byte | number_of_sensors x 1 byte | uptime x 2 byte |`
`error`	`| KO x 1 byte | codice_errore x 1 byte |`

Vengono riportate di seguito alcune considerazioni sulle dimensioni dei payload:

- nelle richieste, per quanto riguarda la codifica del comando, è ampiamente sufficiente 1 byte visto che al momento è possibile effettuare sole 5 tipologie distinte di richieste. Viene predisposto così un formato che ammette l'aggiunta di nuove eventuali future richieste;
- quando si vanno a modificare i valori delle correnti che attraversano i solenoidi, di fatto si modificano delle tensioni applicate ai motori che li controllano. Tali motori, operano su 256 livelli, motivo per cui è sufficiente 1 byte per codificare i valori da fornire alle coppie di pin definiti nel payload di richiesta di `set_solenoid_currents`;
- nei payload di risposta, il primo byte permette di capire se la richiesta è andata a buon fine o meno. KO corrisponde ad un booleano False, OK è un booleano True invece;
- nella risposta di `get_status`, uptime viene codificato su 2 byte così da permettere di sapere il tempo di attività del sistema fino a più di 18 ore di distanza dall'accensione (poi avviene un reset);
- i valori contenuti nei payload di risposta `read_currents` e `get_position` sono numeri in singola precisione, motivo per cui si riservano 4 byte per ciascuno di essi

Risultati

Di seguito, vengono riportati i risultati prodotti da questa terza versione.

Tabella 2.10: Time-testing della versione 3

Comando	t_{TOT} **[**μs**]**
`get_position`	~ 700
`read_currents`	~ 750
`set_solenoid_currents`	~ 400

Si noti come si sono ridotte notevolmente le tempistiche di trasmissione. Questi risultati vanno dunque a giustificare una buona scelta progettuale che ha portato al passaggio dalla formattazione JSON ad una strutturazione binaria dei dati.

2.4 Versione 4

In questa versione, non sono state effettuate modifiche al tipo di packing usato per i dati trasmessi, ma sono state apportate modifiche alla logica di funzionamento del sistema. Fino alla versione precedente, la richiesta `get_position` restituiva i valori grezzi dei campi magnetici sui 3 assi. Ciò implicava la necessità di effettuare adattamento e filtraggio dei valori sul codice Python che esegue sul computer client. Si è pensato dunque di delegare queste operazioni direttamente al Teensy, per due principali ragioni: (1) il codice Arduino è più veloce ad eseguire rispetto a Python, permettendo un campionamento a frequenze maggiori; (2) si riducono i ritardi tra valore campionato e valore effettivamente usato dal controllore.

Poichè il filtraggio e l'adattamento dei valori letti dai sensori avviene ora on-board, a partire da questa versione il payload di risposta di `get_position` necessita di essere adattato come segue, in modo da fornire al programmatore che usa l'interfaccia sia i 3 valori del campo magnetico, sia le rispettive derivate (entrambi già filtrati).

Tabella 2.11: Nuova struttura del payload di risposta di `get_position`

Comando	**Payload**
`get_position`	`| OK x 1 byte | magFieldX x 4 byte | magFieldY x 4 byte | magFieldZ x 4 byte | dMagFieldX x 4 byte | dMagFieldY x 4 byte | dMagFieldZ x 4 byte |`

Gli altri payload rimangono invariati.

Risultati

Siccome le principali migliorie sono state apportate alla logica di funzionamento dell'intero software, e non specificatamente all'impacchettamento dei dati e alla loro trasmissione, non ci sono significative differenze nel test dei tempi rispetto alla versione 3. Le differenze significative si notano nella stabilità del magnete quando viene testato il PD controller. Siccome ora il client non deve più preoccuparsi del filtraggio dei valori letti dai sensori, è possibile eseguire il controllo ed inviare nuovi valori delle correnti dei solenoidi a frequenze maggiori ($\sim 1.2\ kHz$).

2.5 Versione 5

Analizzando meglio il sistema, si è osservato come la codifica COBS non sia necesaria nel presente contesto. Fino alla versione precedente, i dati venivano inviati e ricevuti in maniera sequenziale. Infatti, la codifica COBS era stata utilizzata per poter capire quando una generica sequenza (di dimensioni variabili) terminava. Nel sistema in questione, non è presente alcun flusso di dati continuo in cui si necessita un separatore tra le informazioni bufferizzate. Siccome ad ogni richiesta corrisponde un'immediata risposta con operazioni bloccanti, e poichè sono note a priori le dimensioni dei payload, è possibile effettuare scritture e letture in blocco. É dunque possibile evitare le fasi di codifica e decodifica COBS.

La struttura dei payload rimane uguale a quella adottata nella versione 4, semplicemente non avviene più la codifica/decodifica COBS.
Sono stati inoltre corretti alcuni bug propagati dalle versioni precedenti e sono stati apportati dei miglioramenti sull'utilizzo di alcune funzioni nel codice. Ad esempio, è stato rimosso l'utilizzo di `Sensor.updateData()` dal codice Arduino per inviare la risposta di `get_position`, poichè la lettura dei sensori viene ormai effettuata periodicamente on-board; sul codice dell'interfaccia Python, sono state rimosse le funzioni `flush()` e `reset_input_buffer()` della comunicazione seriale. Queste modifiche hanno permesso di ridurre ulteriormente i tempi di trasmissione che intercorrono tra richiesta da parte del client e relativa risposta del server.

Risultati

Qui vengono riportati i risultati dei test temporali per la versione 5.

Tabella 2.12: Time-testing della versione 5

Comando	t_{TOT} **[**μs**]**
`get_position`	~ 310
`read_currents`	~ 395
`set_solenoid_currents`	~ 310

Si noti come i tempi siano nettamente inferiori rispetto alla versione 1. Infatti, grazie a quest'ultima versione, è stato possibile eseguire il loop di controllo sul client a frequenze di $\sim 1.8\ kHz$, riuscendo a raggiungere la stabilità del sistema per alcuni secondi.

2.6 Considerazioni

Le frequenze con cui opera il sistema non sono ancora ottime al fine di raggiungere la stabilità desiderata. Volendo spostare il controllore su un sistema esterno a quello da controllare, è inevitabile dover affrontare problematiche dovute a ritardi di comunicazione tra i due sistemi. Con la versione 5 del software, è stato possibile trovare un buon compromesso tra organizzazione e strutturazione dei payload e velocità di trasmissione, avvicinandosi sempre di più alle prestazioni del controllo effettuato direttamente on-board.

Capitolo 3

Controllore PD ed esempio di utilizzo dell'interfaccia

Sfruttando l'interfaccia realizzata, è stato implementato un controllore di tipo PD. Grazie all'interfaccia, è stato possibile eseguire il loop di controllo sul client, esternamente al sistema da controllare.

Controllore PD

Il controllore PD (Proportional-Derivative) è una tipologia di controllore lineare utilizzata nei sistemi di controllo per migliorare la reattività e la stabilità del sistema. Esso combina due azioni di controllo:

- Componente proporzionale (P): genera un'azione di controllo proporzionale all'errore tra il valore desiderato (setpoint) e il valore misurato del sistema. Serve a ridurre l'errore in modo diretto e immediato.
- Componente derivativa (D): genera un'azione proporzionale alla derivata dell'errore nel tempo, ovvero alla velocità con cui l'errore sta cambiando. Questa parte introduce un effetto predittivo, contrastando le oscillazioni e migliorando la stabilità.

Il segnale di controllo $u(t)$ generato da un controllore PD è espresso come:

$$u(t) = K_P \cdot e(t) + K_D \cdot \frac{de(t)}{dt}$$

dove:

- $e(t)$ è l'errore, definito come $e(t) = r(t) - y(t)$;
- K_P è il guadagno proporzionale;

- K_D è il guadagno derivativo;
- $r(t)$ è il riferimento;
- $y(t)$ è l'uscita del sistema

Il controllore PD è particolarmente indicato nei sistemi in cui si desidera una risposta rapida senza superamenti eccessivi. Tuttavia, non è in grado di eliminare l'errore stazionario, per il quale sarebbe necessario introdurre un termine integrativo, come avviene nei controllori PID.

Viene mostrato di seguito il codice (Python) relativo all'implementazione del controllore, sfruttando l'interfaccia `MaglevInterface` sviluppata.

```
import time
import sys
from maggy import *

Kp = '''to_be_tuned'''  # guadagno proporzionale
Kd = '''to_be_tuned'''    # guadagno derivativo

PWMALPHA = '''to_be_tuned'''     # costante per il filtro dell'input

CONTROL_FREQUENCY = 1800.0  # Hertz
CONTROL_INTERVAL = 1.0 / CONTROL_FREQUENCY  # seconds

mag_field = {"x": 0.0, "y": 0.0, "z": 0.0}
d_mag_field = {"x": 0.0, "y": 0.0, "z": 0.0}    # derivata

pwm_control_input = {"x": 0, "y": 0}    # x = -x_ ; y = -y_


# logging configuration
logging.basicConfig(level=logging.DEBUG, format="%(asctime)s - %(levelname)s:
↪  %(message)s")


def apply_control_signals():
    maglev.set_solenoid_currents({
        "x+": int(max(0, pwm_control_input["x"])),
        "x-": -int(min(0, pwm_control_input["x"])),
        "y+": int(max(0, pwm_control_input["y"])),
        "y-": -int(min(0, pwm_control_input["y"])),
    })
```

```python
def run_controller():

    previous_control_time = time.time()

    # variabili per controllare l'effettiva frequenza del controllo
    avg_control_time_list = 0
    control_counter = 0

    try:
        while True:
            current_time = time.time()

            if current_time - previous_control_time >= CONTROL_INTERVAL:
                dt2 = current_time - previous_control_time
                previous_control_time = current_time

                values = maglev.get_position()

                mag_field["x"] = values["x"]
                mag_field["y"] = values["y"]
                mag_field["z"] = values["z"]
                d_mag_field["x"] = values["dx"]
                d_mag_field["y"] = values["dy"]
                d_mag_field["z"] = values["dz"]

                if abs(mag_field["z"]) > 18:
                    pwm_control_input["x"] = max(-150, min(150, Kp *
                    ↪  -mag_field["x"] + Kd * -d_mag_field["x"]))
                    pwm_control_input["y"] = max(-150, min(150, Kp *
                    ↪  -mag_field["y"] + Kd * -d_mag_field["y"]))

                    pwm_control_input["x"] = PWMALPHA*pwm_control_input["x"] + (1.0
                    ↪  - PWMALPHA)*prevPwmInputX;
                    pwm_control_input["y"] = PWMALPHA*pwm_control_input["y"] + (1.0
                    ↪  - PWMALPHA)*prevPwmInputY;
                else:
                    pwm_control_input["x"] = 0
                    pwm_control_input["y"] = 0

                # Apply control signals
                apply_control_signals()
```

```
            prevPwmInputX = pwm_control_input["x"]
            prevPwmInputY = pwm_control_input["y"]

            if time.time() % (CONTROL_INTERVAL) < CONTROL_INTERVAL:
                avg_control_time_list += int(1/dt2)
                control_counter += 1

    except KeyboardInterrupt:
        print("\nStopping controller.")
        maglev._close_comm()


    print()
    print(f"Avarage Control Frequency: {avg_control_time_list/control_counter}")
    print()


if __name__ == "__main__":
    try:
        with MaglevInterface('''port''') as maglev:
            print("Keep the magnet away from the system. Sensor calibration in
            ↪  progress...")
            time.sleep(2)  # attendere che il Teensy si inizializzi
            print("Calibration completed.")

            run_controller()    # viene fatto partire il main loop

    except RuntimeError as e:
        print("Runtime error occured!")
        sys.exit(1)
```

Capitolo 4

Considerazioni e future implementazioni

Come introdotto all'inizio del presente elaborato, l'obiettivo era quello di spostare il controllore da on-board ad un dispositivo esterno. Per fare ciò, si è dovuto trovare un modo per far comunicare i due dispositivi. Al fine di standardizzare la comunicazione, e dunque permettere la comunicazione tra *Maggy* e un generico dispositivo, è stata sviluppata un'interfaccia software. Come osservato dai risultati ottenuti nei test temporali delle varie versioni, si è notato come questo metodo di controllo provochi dei rallentamenti nell'esecuzione dello stesso, portando ad instabilità sul sistema da controllare.
Come già illustrato, con la versione 5 dell'interfaccia è stato possibile implementare una prima soluzione in grado di garantire un minimo di stabilità al sistema. Non si tratta ancora della soluzione ottimale però. Per ottenere ulteriori miglioramenti, in futuro ci si potrebbe concentrare sul linguaggio di programmazione e sulla tipologia di connessione fisica tra i sistemi in comunicazione usati.
Per quanto riguarda il linguaggio di programmazione usato, c'è da dire che è stato scelto Python per motivi di semplicità ed intuitività nel suo utilizzo. Siccome il codice deve essere compreso, studiato ed editato dagli studenti, al fine di estendere la didattica sulla Teoria del Controllo anche a chi non ha grandi esperienze di programmazione, Python risultava un'ottima soluzione. Allo stesso tempo però, dal punto di vista delle performance, non garantisce la stessa efficienza e velocità che può fornire un linguaggio compilato come C, ad esempio. Un buon compromesso potrebbe essere quindi la riscrittura dell'interfaccia da Python a C ed utilizzare poi, nel codice di controllo su cui lavorano gli studenti, librerie di Python che permettono l'interfacciamento con codice compilato C. Così facendo, si otterebbe parte del software velocizzato (dato dal passaggio da linguaggio interpretato a compilato) pur mantenendo il codice gestito direttamente dagli allievi in Python.
Chiaramente, focalizzarsi sul linguaggio di programmazione usato, non è l'unica soluzione per migliorare le prestazioni del sistema. Come preannunicato infatti, l'altro aspetto da

non trascurare è quello della tipologia di connessione utilizzata. Al momento, si sta usando comunicazione seriale via USB. Una possibile miglioria potrebbe essere quella di muoversi a cavi Ethernet e comunicazione tramite socket. Queste ultime tecnologie permettono il raggiungimento di velocità di trasmissione dati maggiori.

Inoltre, questa idea di spostare il controllo su un dispositivo esterno a quello da controllare non risulta essere solamente una conveniente soluzione per la didattica, ma anche un valido strumento per poter affinare il controllo. Per meglio dire, l'impiego di telecamere tramite cui fare *computer vision*, potrebbero accurare i dati relativi alla stima della posizione del magnete levitante. Questi dati, andrebbero sincronizzati però con quelli rilevati dai sensori di Hall. Il computer su cui viene eseguito il controllore, risulterebbe dunque un sistema centralizzato a cui collegare le telecamere, oltre al sistema maglev, e su cui fare *sensor fusion*. Chiaramente, per rendere concrete queste idee, sarà prima necessario migliorare le prestazioni in termini di velocità di trasmissione nella comunicazione tra computer e *Maggy*.

Tutto ciò, rende evidente come il sistema sviluppato non sia soltanto adatto per esercitazioni sulla Teoria del Controllo, ma anche una piattaforma altamente versatile e adattabile, capace di evolversi e integrare nuove tecnologie man mano che si rendono disponibili. La possibilità di avere accesso diretto ai meccanismi di controllo, di modificarli e di verificarne in tempo reale gli effetti su un sistema fisico reale, fornisce agli studenti un'esperienza educativa concreta e formativa, spesso difficile da ottenere con i soli strumenti teorici. In un contesto didattico, strumenti come questo risultano fondamentali per colmare il divario tra teoria e pratica: essi permettono agli studenti di osservare e comprendere i concetti fondamentali del controllo automatico, sperimentandone in prima persona le implicazioni. L'adozione di un'interfaccia aperta e modulare, inoltre, consente un alto grado di personalizzazione e stimola la curiosità e l'autonomia nell'apprendimento. In conclusione, il dispositivo realizzato si configura come un valido supporto all'insegnamento, in grado di fornire una base solida su cui costruire percorsi didattici sempre più avanzati e interattivi. Investire nel miglioramento di questo tipo di strumentazione, significa investire nella qualità della formazione degli ingegneri del futuro.

Bibliografia

[1] Wikipedia contributors, *Magnetic levitation*, 2022. indirizzo: https://en.wikipedia.org/w/index.php?title=Magnetic_levitation&oldid=1112700831.

[2] H. A. Engmark e K. T. Hoang, «Modeling and Control of a Magnetic Levitation Platform,» *IFAC-PapersOnLine*, vol. 56, n. 2, pp. 7276–7281, 2023, 22nd IFAC World Congress, issn: 2405-8963. doi: https://doi.org/10.1016/j.ifacol.2023.10.338. indirizzo: https://www.sciencedirect.com/science/article/pii/S240589632300705X.

[3] Wikipedia contributors, *Earnshaw's theorem*, 2025. indirizzo: https://en.wikipedia.org/wiki/Earnshaw%27s_theorem.

[4] Wikipedia contributors, *USB communications device class*, 2025. indirizzo: https://en.wikipedia.org/wiki/USB_communications_device_class.

[5] H.A. Engmark, *Take-home Maglev lab*, 2022. indirizzo: https://github.com/Hansolini/Take-home-Maglev-lab.